

```

#include <iostream>
#include <cstring>
#include <cstdlib>

using namespace std;

class DateUtils {
public:
    static const int days[13]; // Declare a static constant array to hold the number of days in each month

    // Static method to check if a year is a leap year
    static bool leap(int y) {
        return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0); // Return true if year is a leap year
    }

    // Static method to get the date from a given day number
    static void get_date(int day) {
        int y, m;
        // Calculate the year by subtracting the days of each year until remaining days fit in the
        current year
        for (y = 1700; day > 365 + leap(y); y++)
            day -= 365 + leap(y);

        // Calculate the month by subtracting the days of each month until remaining days fit in the
        current month
        for (m = 1; day > days[m] + (m == 2 && leap(y)); m++)
            day -= days[m] + (m == 2 && leap(y));

        // Print the date in mm/dd/yyyy format
        cout << m << "/" << day << "/" << y;
    }

    // Static method to convert date in yyymmdd format to the number of days since 1700
    static int f(int x) {
        int i, tmp, sum;
        tmp = x / 10000; // Extract the year from the date
        sum = 0;

        // Sum the days for each full year since 1700
        for (i = 1700; i < tmp; i++) {
            sum += 365;
            if (leap(i)) // Add an extra day for leap years
                sum++;
        }

        tmp = x % 10000 / 100; // Extract the month from the date
        for (i = 1; i < tmp; i++) {
            sum += days[i];
            if (i == 2 && leap(x / 10000)) // Add an extra day for leap years in February
                sum++;
        }

        return sum + x % 100; // Add the days of the current month
    }
};

// Initialize the static constant array with the number of days in each month
const int DateUtils::days[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

```

```

class QuoteManager { // Define a class named QuoteManager
private:
    int n, m, a, b, cases, no_quotes; // Variables to store input and state information
    int st[101], ed[101]; // Arrays to store start and end dates for quotes
    char c[147000]; // Array to keep track of days (0 or 1 or 2)

public:
    // Constructor to initialize member variables and array
    QuoteManager() : n(0), m(0), cases(0), no_quotes(1) {
        memset(c, 0, sizeof(c)); // Initialize array c with zeros
    }

    // Method to fill the array c with value x from day a to day b
    void fill(int a, int b, int x) {
        int fa = DateUtils::f(a); // Convert start date to the number of days since 1700
        int fb = DateUtils::f(b); // Convert end date to the number of days since 1700
        memset(&c[fa], x, fb - fa + 1); // Fill the range from fa to fb with value x
    }

    // Method to read input data and process each case
    void readInput() {
        while (cin >> n >> m && (n + m)) { // Read number of quotes and queries until both are zero
            memset(c, 0, sizeof(c)); // Clear the array c

            for (int i = 0; i < n; i++)
                cin >> st[i] >> ed[i]; // Read start and end dates for each quote

            for (int i = 0; i < m; i++) {
                cin >> a >> b; // Read the dates to fill with 2
                fill(a, b, 2); // Fill the range with 2
            }

            for (int i = 0; i < n; i++)
                fill(st[i], ed[i], 1); // Fill the array with 1 for the quotes

            cout << "Case " << ++cases << ":\n"; // Print the case number
            no_quotes = 1; // Assume no quotes are required

            for (int i = 0; i < 147000; i++) {
                if (c[i] > 1) { // If the day has a value greater than 1
                    no_quotes = 0; // Set no_quotes flag to 0
                    DateUtils::get_date(i); // Get the start date
                    if (c[i + 1] < 2)
                        cout << "\n"; // Print the date if the next day is not > 1
                    else {
                        cout << " to ";
                        int j;
                        for (j = i; c[j] > 1; j++); // Find the end of the range
                        DateUtils::get_date(j - 1); // Get the end date
                        cout << "\n";
                        i = j; // Skip to the end of the range
                    }
                }
            }
            if (no_quotes)
                cout << "No additional quotes are required.\n"; // Print if no quotes are required
        }
    }
};

```

```
int main() {  
    QuoteManager manager; // Create an instance of QuoteManager  
    manager.readInput(); // Read input and process quotes  
  
    return 0; // Return 0 to indicate successful execution  
}
```